NEXUS — Manual técnico y preparación para el examen H3.

1. **Sección I — Arquitectura y POO (Tu código real):**

    - Personaje **(Clase Abstracta):** Define Nombre, Salud, Energia y el método abstracto Analizar().

    - Cadete**,** Robot**,** Enemigo **(Clases Derivadas):** Heredan de Personaje y redefinen/especializan Analizar() (Polimorfismo) y agregan sus propios atributos (Escudo, Peligro, FuerzaAtaque, etc.).

    - Explorador**:** Hereda directamente de Cadete.

    - Objeto **/** TipoObjeto **(Enum):** Manejo de inventario.

    - Mapa**,** ConexionNexus**,** NexusSistema**,** Mision**:** Clases del sistema para mapas 2D, sockets TCP y flujo de menú.

2. **Sección II — Fundamentos:**

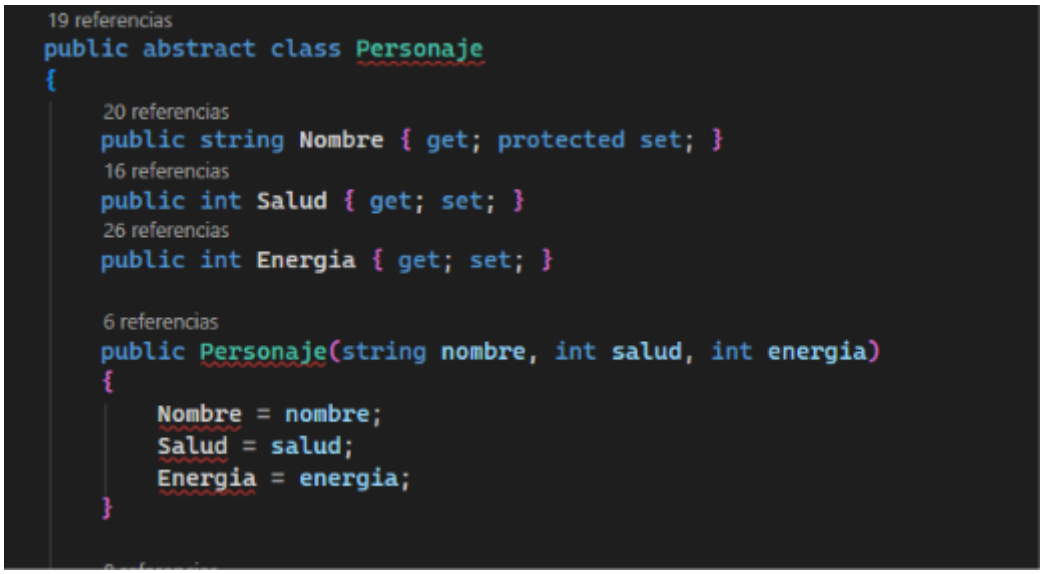
    - **Atributos y Propiedades:** En la clase Cadete (líneas 31 a 40), donde tienes propiedades con distintos modificadores como public List<Objeto> Inventario { get; private set; }.



```
public class Cadete : Personaje
{
    2 referencias
    public int Edad { get; private set; }
    2 referencias
    public string Realidad { get; private set; }
    14 referencias
    public int Estabilidad { get; private set; }
    4 referencias
    public int Peligro { get; private set; }
    4 referencias
    public int DatosIris { get; private set; }

    10 referencias
    public int Escudo { get; set; }
    4 referencias
    public int XP { get; set; }
    2 referencias
    public int ModuloAtaque { get; set; }
    6 referencias
    public List<Objeto> Inventario { get; private set; }
```

- **Constructores y Sobrecarga:** Constructor base de Personaje(string, int, int) y el constructor de Cadete que llama a : base(nombre, 100, energia).



```
19 referencias
public abstract class Personaje
{
    20 referencias
    public string Nombre { get; protected set; }
    16 referencias
    public int Salud { get; set; }
    26 referencias
    public int Energia { get; set; }

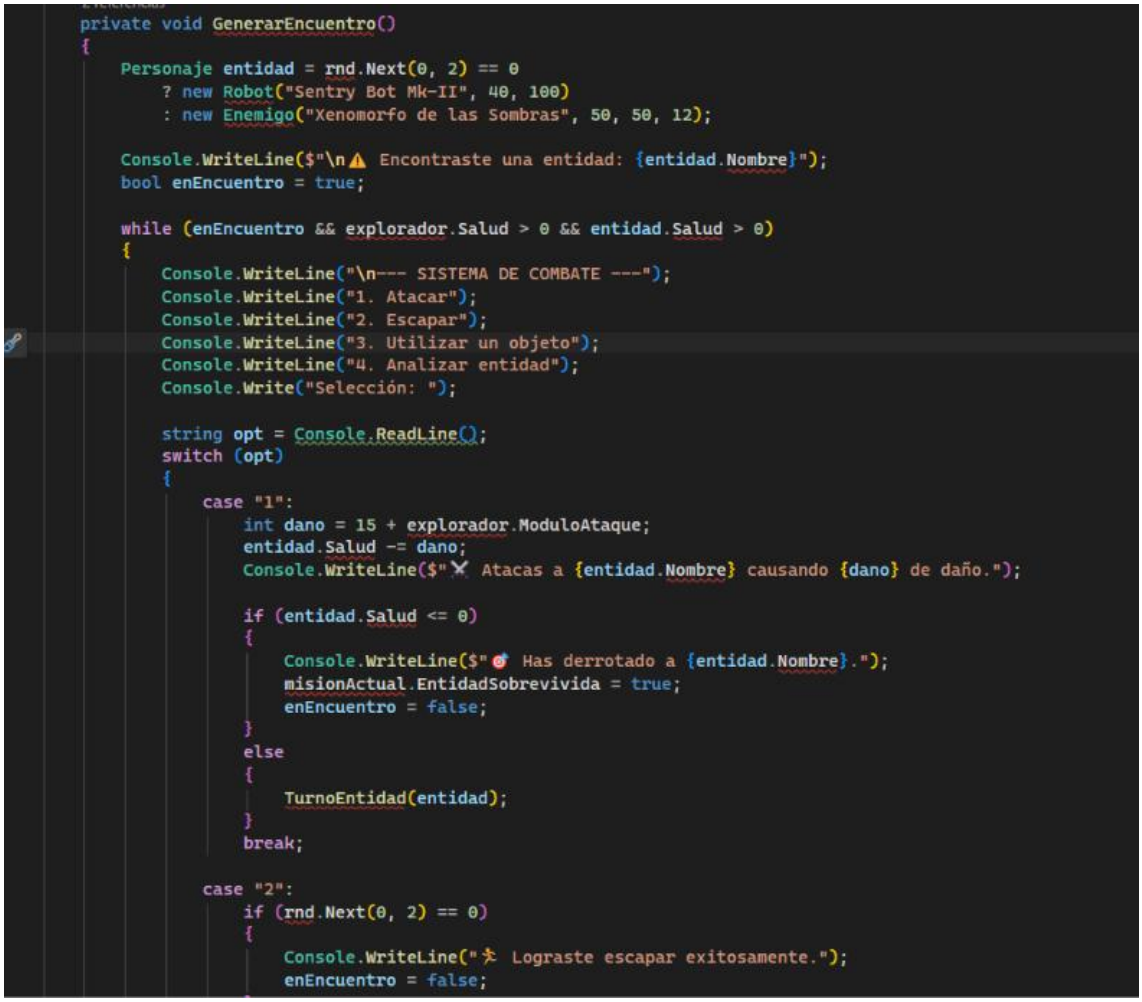
    6 referencias
    public Personaje(string nombre, int salud, int energia)
    {
        Nombre = nombre;
        Salud = salud;
        Energia = energia;
    }
```

- this **/** base**:** Uso explícito de base en los constructores de Cadete, Robot y Enemigo.

3. **Sección III — Herencia y Polimorfismo (Ubicación exacta):**

- **Método Abstracto:** public abstract void Analizar(); en Personaje.

- **Polimorfismo Real:** En el método GenerarEncuentro() (líneas 439 a 500) creas un Personaje entidad = new Robot(...) o new Enemigo(...) y luego ejecutas entidad.Analizar(). Según el tipo real en memoria, C# ejecuta el Analizar() del Robot o del Enemigo.



```
private void GenerarEncuentro()
{
    Personaje entidad = rnd.Next(0, 2) == 0
        ? new Robot("Sentry Bot Mk-II", 40, 100)
        : new Enemigo("Xenomorfo de las Sombras", 50, 50, 12);

    Console.WriteLine($"\n⚠️ Encontraste una entidad: {entidad.Nombre}");
    bool enEncuentro = true;

    while (enEncuentro && explorador.Salud > 0 && entidad.Salud > 0)
    {
        Console.WriteLine("\n--- SISTEMA DE COMBATE ---");
        Console.WriteLine("1. Atacar");
        Console.WriteLine("2. Escapar");
        Console.WriteLine("3. Utilizar un objeto");
        Console.WriteLine("4. Analizar entidad");
        Console.Write("Selección: ");

        string opt = Console.ReadLine();
        switch (opt)
        {
            case "1":
                int dano = 15 + explorador.ModuloAtaque;
                entidad.Salud -= dano;
                Console.WriteLine($"⚔️ Atacas a {entidad.Nombre} causando {dano} de daño.");

                if (entidad.Salud <= 0)
                {
                    Console.WriteLine($"🎯 Has derrotado a {entidad.Nombre}.");
                    misionActual.EntidadSobrevivida = true;
                    enEncuentro = false;
                }
                else
                {
                    TurnoEntidad(entidad);
                }
                break;

            case "2":
                if (rnd.Next(0, 2) == 0)
                {
                    Console.WriteLine("🏃 Lograste escapar exitosamente.");
                    enEncuentro = false;
```



```
                else
                {
                    TurnoEntidad(entidad);
                }
                break;

            case "2":
                if (rnd.Next(0, 2) == 0)
                {
                    Console.WriteLine("🏃 Lograste escapar exitosamente.");
                    enEncuentro = false;
                }
                else
                {
                    Console.WriteLine("❌ Escape fallido. La entidad bloquea tu salida.");
                    TurnoEntidad(entidad);
                }
                break;

            case "3":
                GestionarInventario();
                break;

            case "4":
                entidad.Analizar();
                break;

            default:
                Console.WriteLine("⚠️ Opcion invalida. La entidad aprovecha tu duda.");
                TurnoEntidad(entidad);
                break;
        }
    }
}

6 referencias
```

- **Ejemplo Complementario (Interfaz):** Dado que en tu archivo no utilizaste la palabra clave interface, en el manual aclaramos que la interfaz IGestionable es un **ejemplo complementario** exigido por la guía para demostrar el concepto teórico.

4. **Sección IV — Colecciones y LINQ:**

- **Listas Genéricas (**List<T>**):** Tu lista de inventario List<Objeto> en la clase Cadete.

- **Ejemplo Complementario de LINQ:** Como en tu código recorres el inventario usando bucles for, incluimos en el manual la consulta equivalente con LINQ (Where, Select, OrderBy) señalándola como mejora complementaria.

Versión final ajustada a TU CÓDIGO

==========

NEXUS — MANUAL TÉCNICO Y PREPARACIÓN PARA EL EXAMEN H3

==========

1. RESPUESTA AL ABP: ESTRATEGIA DE ESTUDIO Y RENDIMIENTO EN H3

La preparación para el examen del Hito 3 se basa en la comprensión estructural y práctica del código fuente real del sistema NEXUS (namespace NEXUS en Program.cs).

I. ARQUITECTURA Y PROGRAMACIÓN ORIENTADA A OBJETOS

---

1. Propósito General:

NEXUS es un sistema interactivo de consola desarrollado en C# para la simulación de exploración, combate y cartografía en un entorno espacial.

2. Estructura del Proyecto y Clases (Program.cs):

- Personaje (Clase Abstracta): Base de las entidades del sistema (Salud, Energia, Analizar()).

- Cadete / Explorador (Clase Derivada): Representa al jugador con sistema de peligro, escudo e inventario.

- Robot / Enemigo (Clases Derivadas): Entidades con comportamientos específicos de combate y defensa.

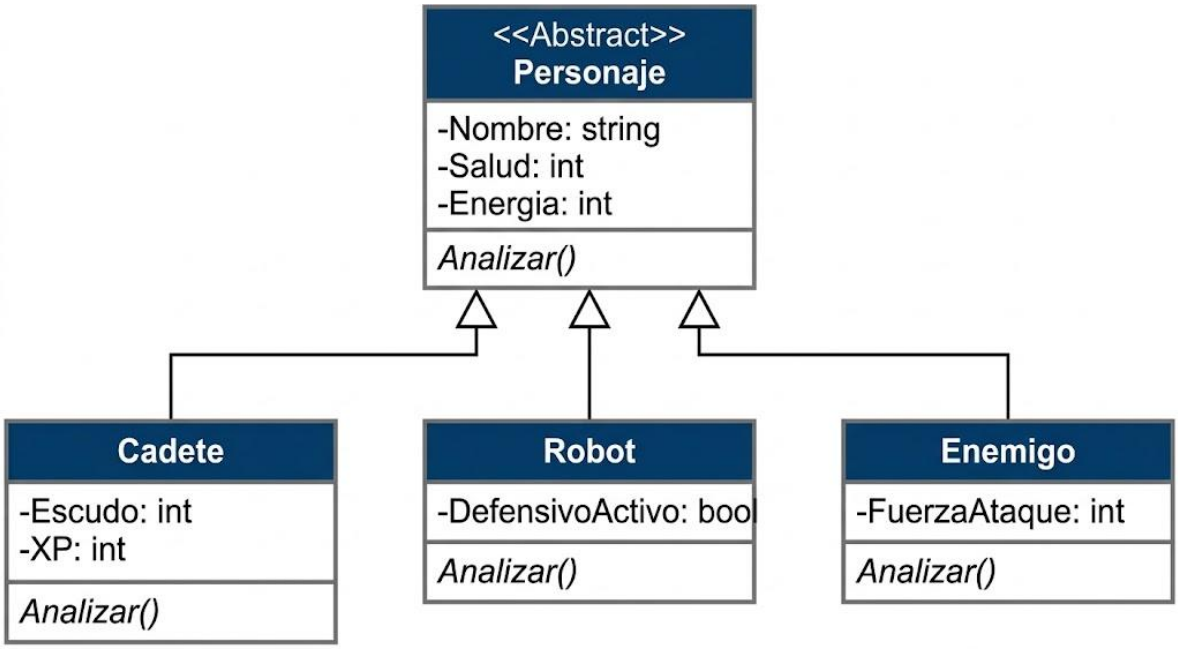
- Objeto / TipoObjeto (Enum): Entidades de inventario (Energía, Escudo, Analizador).

- Mapa: Matriz 2D (13x29) para cartografía y movimiento (W/S/A/D).

- ConexionNexus: Manejo de sockets TCP (TcpListener / TcpClient) para comunicación remota.

- NexusSistema: Controlador principal del flujo de la sesión y menús.

3. Diagrama de Clases:



II. FUNDAMENTOS DE CLASES Y OBJETOS

---

1. Instanciación y Encapsulamiento:

En la clase Cadete, las propiedades usan getters públicos y setters privados para resguardar la integridad de los datos:

```
public int Estabilidad { get; private set; }

public List<Objeto> Inventario { get; private set; }
```

2. Uso de base y constructores:

La clase Cadete utiliza la palabra clave 'base' para inicializar los atributos de la clase abstracta Personaje:

```
public Cadete(string nombre, int edad, string realidad, int energia, int estabilidad)
    : base(nombre, 100, energia) { ... }
```

III. HERENCIA, POLIMORFISMO E INTERFACES

---

1. Polimorfismo Real en NEXUS:

En el método GenerarEncuentro(), el sistema instancia un objeto de tipo Robot o Enemigo guardándolo en una referencia base de tipo Personaje:

```
Personaje entidad = rnd.Next(0, 2) == 0
    ? new Robot("Sentry Bot", 40, 100)
    : new Enemigo("Xenomorfo", 50, 50, 12);
entidad.Analizar(); // Llama dinámicamente al método del Robot o del Enemigo.
```

2. Ejemplo Complementario (Interfaz):

(Nota: Concepto complementario exigido por la guía de evaluación)

```
public interface IGestionable { void Procesar(); }
```

IV. COLECCIONES Y LINQ

---

1. Listas Genéricas:

NEXUS gestiona los objetos recolectados mediante List<Objeto>:

```
explorador.Inventario.Add(new Objeto(" 🔋 Célula de energía",
TipoObjeto.Energia, 25));
```

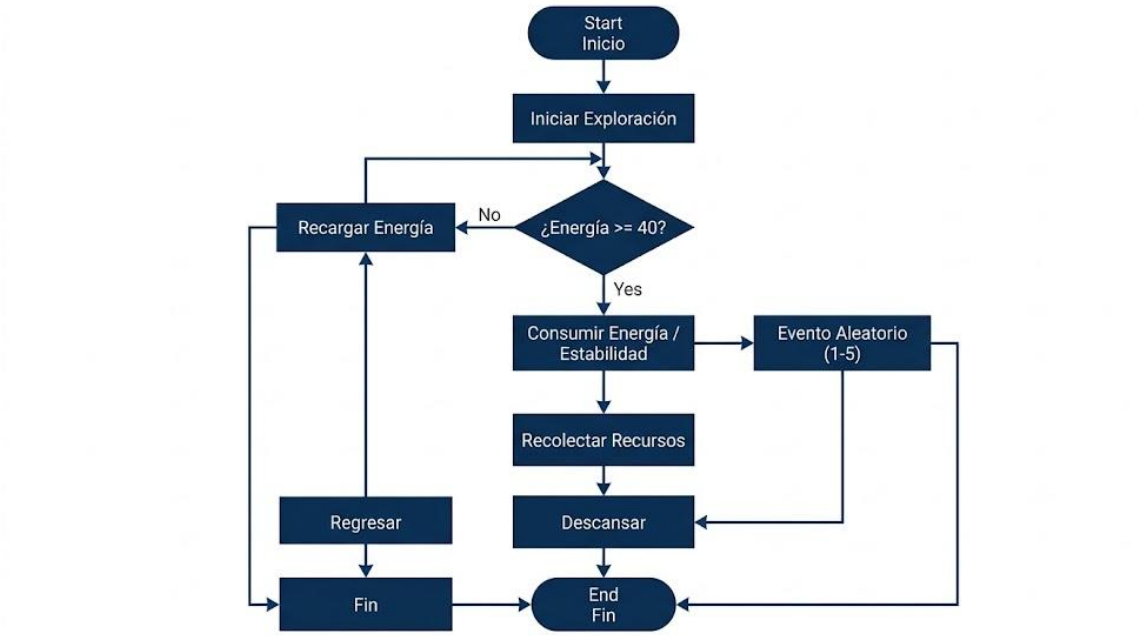
2. Ejemplo Complementario (Consulta LINQ equivalente):

```
var objetosEnergia = explorador.Inventario
    .Where(o => o.Tipo == TipoObjeto.Energia)
    .OrderByDescending(o => o.Valor);
```

V. ANÁLISIS DEL FUNCIONAMIENTO

---

1. Flujograma de Exploración:



2. Seguimiento de Estado:

Instanciación: Cadete c = new Cadete("Joel", 20, "Neo Cochabamba", 100, 80); (Peligro = 10)

Acción: c.Explorar(); -> Energía baja a 85, Estabilidad a 73 -> Peligro se recalcula automáticamente en la Heap (Peligro = 21).

Colecciones y LINQ

Listas Genéricas (`List<T>`)

En la clase `Cadete` de NEXUS, la recolección de ítemes se gestiona mediante una lista genérica dinámica:

```csharp
public List<Objeto> Inventario { get; private set; } = new List<Objeto>();
```

```

* **Manipulación y Recorrido:** Se agregan objetos con `.Add()` al explorar la matriz o derrotar entidades y se recorren de forma tradicional con bucles `for` o `foreach` para mostrar el menú de interacción.

Expresiones Lambda

Son funciones anónimas concisas utilizadas para escribir lógica en línea dentro de consultas. La sintaxis `(parámetros) => expresión` se emplea para definir predicados de filtrado y transformación sobre la lista de objetos.

* *Ejemplo:* `x => x.Tipo == TipoObjeto.Energia` (evalúa si el objeto es de tipo energía).

---

#### 3. Consultas LINQ: `Where`, `Select`, `OrderBy` y Otras

*(Nota: Código complementario aplicado sobre el inventario real de NEXUS para la preparación del examen H3)*.

```csharp
using System.Linq;

// Filtrado, Ordenamiento y Transformación
var inventarioFiltrado = explorador.Inventario
    .Where(o => o.Valor > 10)              // Filtrado (Where)
    .OrderByDescending(o => o.Valor)       // Ordenamiento (OrderBy)
```
```

.Select(o => new { o.Nombre, o.Valor }); // Transformación / Proyección (Select)

```

* **`Where` (Filtrado):** Selecciona únicamente los elementos que cumplen una condición booleana.

* **`OrderBy` / `OrderByDescending` (Ordenamiento):** Organiza la lista en función del valor de una propiedad.

* **`Select` (Transformación):** Proyecta los objetos en un nuevo formato o tipo anónimo seleccionando solo las propiedades necesarias.

* **Otras utilizadas (`FirstorDefault`, `Count`, `Sum`):**

```csharp

int totalEnergia = explorador.Inventario

.Where(o => o.Tipo == TipoObjeto.Energia)

.Sum(o => o.Valor); // Suma el valor total de energía recuperable

```

Comparación entre Consultas LINQ y Recorridos Tradicionales

| Criterio | Recorrido Tradicional (`foreach` / `for`) | Consulta LINQ (`Where` / `Select`) |

| --- | --- | --- |

| **Enfoque** | **Imperativo:** Se debe especificar *cómo* iterar, acumular y filtrar los datos paso a paso.
```

| **Declarativo:** Se especifica *qué* datos se desean obtener usando expresiones lambda. |

| **Legibilidad** | Requiere múltiples líneas de código, condicionales (`if`) y listas auxiliares.

| Compacto, expresivo y fácil de mantener en una sola cadena de métodos. |

| **Ejecución** | Se evalúa de forma inmediata elemento por elemento durante el bucle.

| **Evaluación diferida (*Deferred Execution*):** La consulta no se ejecuta hasta que se itera sobre el resultado o se convierte a lista (`ToList()`). |

VI. PREPARACIÓN PARA EL EXAMEN (6 PREGUNTAS UNIVERSITARIAS)

---

1. ¿Por qué Personaje es abstracta y qué pasaría si se intenta hacer 'new Personaje()'?

R: Ocurre un error de compilación (CS0144). Las clases abstractas sirven como modelos base y no pueden instanciarse directamente.

2. Explique cómo funciona el polimorfismo en 'entidad.Analizar()'.

R: Permite tratar a Robot y Enemigo de forma uniforme desde la clase base Personaje, ejecutando en tiempo de ejecución la versión 'override' correspondiente.

3. En la clase Cadete, ¿qué ventaja ofrece 'public List<Objeto> Inventario { get; private set; }'?

R: Implementa encapsulamiento. Cualquiera puede leer la lista, pero solo la clase Cadete puede reasignar la referencia de la lista.

4. ¿Cómo evita la clase Mapa que el jugador se salga de los límites de la matriz?

R: Mediante validación condicional previa en Mover(): si (nf < 0 || nf >= 13 || nc < 0 || nc >= 29) ignora el movimiento.

5. ¿Qué diferencia existe entre List<Objeto> y un arreglo 'Objeto[]'?

R: List<T> es una colección dinámica en memoria que permite agregar (Add) y eliminar (RemoveAt) elementos flexiblemente.

6. Corrija el error: En 'Cadete' se intenta modificar directamente 'Salud' cuando recibe daño pero esta es de la clase base.

R: 'Salud' tiene setter público en 'Personaje', lo que permite modificarlo, pero lo ideal es encapsularlo con un método como RecibirDaño(int cantidad).

DIAGNÓSTICO DE APRENDIZAJE Y APORTES PERSONALES

(PROMPT ESPEJO — EVALUACIÓN DEL PROCESO EN LA MISIÓN NEXUS: DE CÓDIGO A CONOCIMIENTO)

1. Fortalezas demostradas

- **Verificación activa de la infraestructura:** Demostraste iniciativa al consultar directamente sobre la visibilidad del repositorio (público vs. privado) y las pruebas de acceso mediante ventanas de incógnito.

- **Comprensión de la organización de proyectos Git:** Identificaste correctamente que el código de la consola se encontraba en el archivo principal Program.cs en la raíz del proyecto, cuestionando justificadamente por qué el borrador inicial hacía referencia a una carpeta src/.

- **Criterio visual para la documentación:** Notaste la necesidad de presentar los diagramas (diagrama de clases UML y flujograma de procesos) en formato visual/imagen en lugar de simple texto plano, elevando la calidad del informe técnico.

- **Claridad en la estructura del entorno C#:** Aportaste la totalidad del código fuente real de tu proyecto en C# (namespace NEXUS), permitiendo alinear los conceptos de POO (clase abstracta Personaje, derivadas Cadete, Robot, Enemigo) con la implementación real del software.

2. Falencias observadas

- **Dependencia inicial de la plantilla:** Al principio aceptaste modelos teóricos genéricos (EntidadBase.cs, GestorNEXUS.cs) antes de solicitar

que la documentación se adaptara específicamente a tu archivo Program.cs.

- **Dudas sobre la jerarquía de archivos en Git:** Surgieron dudas recurrentes respecto a dónde debía alojarse la documentación en PDF (dentro de la carpeta docs/) en relación con el archivo ejecutable de C# y el README.md.

- **Omisión inicial de construcciones teóricas no implementadas:** No habías identificado qué conceptos exigidos por la rúbrica (como interfaces interface o consultas avanzadas LINQ) no estaban presentes de forma explícita en el código original de C#.

3. Diagnóstico del Hito 3

Concepto de Programación II	**Clasificación de Dominio**	**Justificación basada en la conversación**
POO, Clases y Objetos	**Comprensión Demostrada**	Definiste clases claras para el dominio del juego (Cadete, Robot, Enemigo, Objeto) instanciándolas en memoria mediante NexusSistema.
Constructores y Encapsulamiento	**Comprensión Demostrada**	Uso correcto de getters públicos con setters privados (public List<Objeto> Inventario { get; private set; }) y llamadas a constructores base (: base(...)).
Herencia y Polimorfismo	**Comprensión Demostrada**	Aplicaste la clase abstracta Personaje con el método abstracto Analizar() y redefiniciones override en las clases derivadas, logrando

Concepto de Programación II	Clasificación de Dominio	Justificación basada en la conversación
		resolución dinámica en GenerarEncuentro().
Interfaces y Contratos	**Comprensión Parcial**	Tu código base no incluía palabras clave interface, por lo que se requirió el desarrollo de ejemplos complementarios (IGestionable) para cubrir la exigencia teórica.
Listas Genéricas (List<T>**)**	**Comprensión Demostrada**	Manejo de listas dinámicas en memoria para el inventario de objetos (List<Objeto>), permitiendo adición, eliminación e inspección.
Consultas LINQ	**Comprensión Parcial**	El código original realiza recorridos mediante bucles imperativos tradicionales (for/foreach). Se formularon ejemplos complementarios con Where, Select y OrderBy para el análisis.

4. Mi forma de aprender

Demostraste un enfoque **práctico, visual e iterativo**:

- Investigas probando directamente en el entorno de desarrollo y en la interfaz de GitHub.

- Solicitas validaciones continuas sobre capturas de pantalla para asegurarte de que lo que tienes en pantalla coincide con los requisitos.

- Corriges desalineaciones inmediatamente cuando detectas que la documentación difiere de la realidad de tu código.

5. Plan de estudio (5 Conocimientos a reforzar antes del examen)

1. **Sintaxis manual de Interfaces (**interface**)**

    - *Ejercicio:* Crea una interfaz IAtacable con el método void RecibirDaño(int cantidad) e impleméntala explícitamente en Cadete y Enemigo.

2. **Consultas LINQ con Expresiones Lambda**

    - *Ejercicio:* Reescribe el método de filtrado de inventario de Cadete utilizando Inventario.Where(x => x.Tipo == TipoObjeto.Energia).ToList() a mano en papel.

3. **Identificación de Memoria (Stack vs. Heap)**

    - *Ejercicio:* Dibuja el diagrama de memoria de la ejecución de Cadete explorador = new Cadete(...) identificando dónde se aloja la referencia y dónde el objeto real.

4. **Manejo de Métodos Abstractos vs. Virtuales**

    - *Ejercicio:* Explica la diferencia entre public abstract void Analizar() y un método public virtual void ActualizarPeligro() redactando qué ocurre si la derivada omite la palabra override.

5. **Lectura y Trazabilidad de Excepciones**

    - *Ejercicio:* Analiza el bloque try/catch en la clase ConexionNexus y predice qué excepción captura SocketException cuando falla la conexión por TCP.

6. Conclusión

Has demostrado capacidad para construir y defender un sistema en C# que aplica correctamente los pilares esenciales de la POO (Herencia, Encapsulamiento y Polimorfismo) e integración con Git/GitHub. Para el examen presencial del Hito 3, debes enfocar tu repaso en redactar sintaxis en papel sin ayuda del autocompletado del IDE, asegurando el dominio de las expresiones Lambda/LINQ y la definición de interfaces.